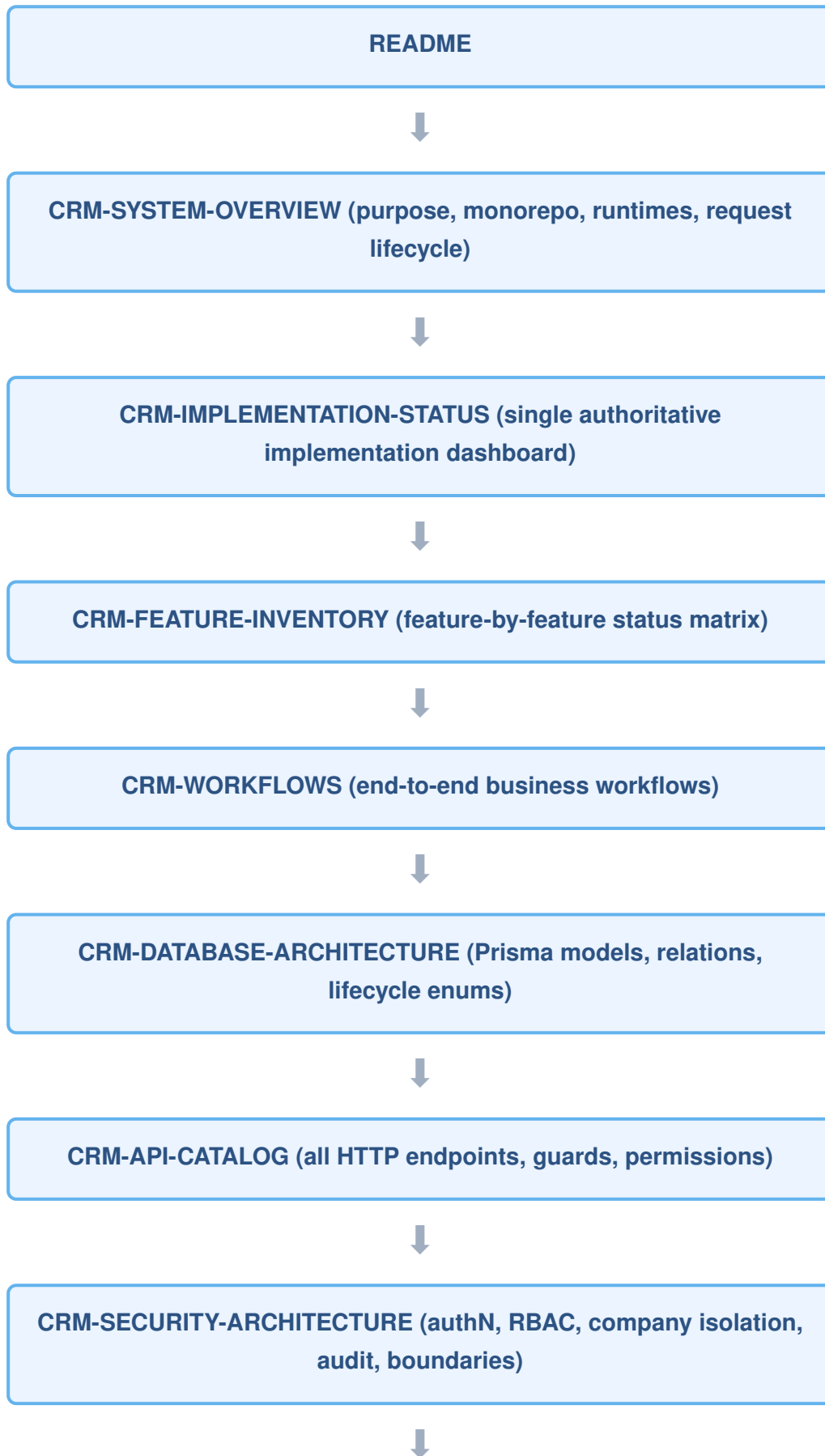


RRH-CRMS — System Documentation (Master Index)

Authoritative, **repository-verified** reference for the **RRH-CRMS** monorepo ([RRH PWA](#)). Every claim is traceable to source files (paths are relative to the repo root). This package is the **current-state technical reference**, and it strictly separates **CURRENT IMPLEMENTATION** from **APPROVED ARCHITECTURE**, **FUTURE / V2**, and **OUT OF SCOPE**.

How to read this documentation (recommended order)



CRM-AI-ARCHITECTURE (Phase 17-A AI search boundary, JSON contract, provider abstraction)



CRM-WEBSITE-INTEGRATION (website ↔ CRM responsibility boundary)



CRM-V2-ROADMAP (deferred / future / out-of-scope)

Document index

#	Document	Covers
1	CRM-SYSTEM-OVERVIEW	Purpose & scope, monorepo layout, runtimes, request lifecycle, conventions
2	CRM-IMPLEMENTATION-STATUS	Single status dashboard across every domain (legend applied)
3	CRM-FEATURE-INVENTORY	Feature matrix: backend / frontend / DB / API / tests
4	CRM-WORKFLOWS	Authentication, Lead, Customer, Property, Site Visit, Booking, Payment, Collections, Documents, KYC, Task/SLA, Complaints, Expense Refund, Reporting
5	CRM-DATABASE-ARCHITECTURE	Every active Prisma model, relations, lifecycle enums, relationship map
6	CRM-API-CATALOG	Internal, Public, Auth, AI, Integration endpoints with guards/permissions
7	CRM-SECURITY-ARCHITECTURE	authN (JWT + refresh rotation), RBAC, <code>can()</code> engine, company isolation, audit, uploads, public API; implemented vs missing
8	CRM-AI-ARCHITECTURE	Phase 17-A boundary, AI Search workflow, SearchIntent JSON contract, provider abstraction, security, limitations, V2 boundaries
9	CRM-WEBSITE-INTEGRATION	Website ↔ CRM responsibility boundary, public APIs, AI-search flow, brand separation

#	Document	Covers
10	CRM-V2-ROADMAP	Deferred functionality, V2 boundaries, out-of-scope exclusions

Status legend (used throughout)

Icon	Meaning
	IMPLEMENTED (verified in source)
	PARTIALLY IMPLEMENTED
	NOT IMPLEMENTED
	FUTURE / V2
	OUT OF SCOPE
	UNKNOWN / HUMAN REVIEW REQUIRED

Evidence discipline

- The repository is the **source of truth**: code, routes, services, Prisma schema, permissions, workflows, validation, business logic, tests, configuration.
- Roadmaps / existing reports are **supporting evidence only**.
- If documentation says X but code does not implement X → marked **NOT IMPLEMENTED**.
- If code implements something undocumented → marked **IMPLEMENTED / UNDOCUMENTED**.
- If a claim cannot be conclusively verified →  **INSUFFICIENT REPOSITORY EVIDENCE — HUMAN REVIEW REQUIRED**.

CRM-SYSTEM-OVERVIEW

Executive and technical overview of the **RRH-CRMS** monorepo, verified against the repository at commit [93cf058](#) (main, ahead of origin).

1. Purpose & Scope

RRH-CRMS is a **multi-tenant, role-based real-estate CRM** plus a **Progressive Web App (PWA)** serving the RRH Group brands (**RADHA_REAL_HOMES, SONTHILLU**). It covers the full internal sales operations pipeline: lead management, opportunity pipeline, customer & KYC, project & property inventory, site visits, bookings, payments & installments, expense refunds, complaint management, document management, employee HR/attendance, daily targets, task/SLA, performance scoring, notifications, audit, and analytics. It also exposes **public APIs** for website integration and a **customer-portal integration** layer (Phase 11) with a *future* customer portal.

Phase 17-A adds an **AI natural-language → structured-search-intent** boundary. Per the approved architecture this AI is an **interpretation layer only**: it converts natural language into a structured **SearchIntent**; the CRM performs all deterministic matching, scoring, ranking, and business decisions.

2. Monorepo Layout (verified)

```

RRH PWA/                                     # repo root
├─ apps/
│  └─ api/                                   # Express + TypeScript REST API (CommonJS, rootDir src)
│     └─ src/
│        ├── server.ts                     # app bootstrap, middleware, route mounting
│        ├── authz/                       # can() engine + data-scope builders
│        ├── middleware/                  # auth (JWT/service token), authz, validate, rate limiter, correlation
│        ├── policies/                   # object-level authorization policies
│        ├── routes/                     # Express routers (one per domain)
│        ├── services/                   # business services (+ ai/ Phase 17-A)
│        ├── utils/                      # jwt, crypto, matchingEngine, hierarchy, slugify, qr, etc.
│        └─ workflows/                   # workflow engine + domain workflows
│     └─ dist/                           # compiled output
│  └─ web/                               # Vite + React + TypeScript PWA
│     └─ src/                            # components/, context/, hooks/, pages via React Router
├─ packages/
│  └─ shared/                            # canonical Roles, Permissions, RolePermissionsMatrix, zod schemas
├─ prisma/
│  ├── schema.prisma                     # source of truth for the data model
│  ├── seed.ts                          # seeds Roles/Permissions/branches/company
│  └─ migrations/                       # 20+ migration folders (MySQL)
├─ tests/
│  ├── api/                             # Jest + ts-jest integration/unit tests
│  ├── fixtures/                        # testUsers.ts
│  └─ utils/                            # authHelpers.ts
├─ scripts/                             # DB push/verify/migration scripts
├─ docs/                                # roadmap + transformation reports (context)
├─ uploads/                             # server-side uploads (expense proofs, property images)
├─ QR/                                  # QR assets
├─ .env / .env.example / .env.test
└─ package.json / tsconfig.json / jest.config.js / playwright.config.ts

```

3. Runtimes & Toolchain (verified)

Layer	Technology	Source evidence
API backend	Node.js + TypeScript + Express 4	<code>apps/api/src/server.ts</code> , <code>package.json</code>
ORM	Prisma 5 (schema-first, MySQL provider)	<code>prisma/schema.prisma</code> (<code>datasource db</code> , <code>provider = "mysql"</code>)
Validation	Zod (shared schemas + route-level <code>validateRequestBody</code>)	<code>packages/shared/src/index.ts</code> , <code>apps/api/src/middleware/validate.ts</code>

Layer	Technology	Source evidence
Auth	JWT access tokens (Bearer) + httpOnly refresh cookie + refresh rotation	<code>apps/api/src/utils/jwt.ts</code> , <code>apps/api/src/routes/auth.ts</code>
Authorization	Central <code>can()</code> engine + <code>requireAuthz</code> middleware + object policies + data scopes	<code>apps/api/src/authz/*</code> , <code>apps/api/src/middleware/authz.ts</code> , <code>apps/api/src/policies/*</code>
Shared constants	Roles, Permissions, RolePermissionsMatrix, zod schemas	<code>packages/shared/src/index.ts</code>
API tests	Jest + ts-jest + supertest	<code>jest.config.js</code> , <code>tests/api/*</code>
E2E	Playwright (declared)	<code>playwright.config.ts</code> , <code>tests/*.spec.ts</code>
Frontend	Vite + React + TypeScript PWA	<code>apps/web/src/main.tsx</code> , <code>App.tsx</code>
Extra deps	<code>openai</code> , <code>@openrouter/sdk</code> (declared, unused by AI wire-up), <code>sharp</code> , <code>bcryptjs</code> , <code>express-rate-limit</code>	<code>package.json</code>

4. Request Lifecycle (API)

1. **Proxy/security** — `app.set('trust proxy', 1)` ; `helmet()` ; CORS (credentials); `cookie-parser` ; JSON body parser.
2. **Rate limit** — domain-specific limiters (`publicReadLimiter` , `publicWriteLimiter` , `loginRateLimiter` , etc.) in `apps/api/src/middleware/rateLimiter.ts` .
3. **Authenticate** — one of:
 4. `authenticateToken` (employee JWT) → `req.user: TokenPayload` ;
 5. `authenticateServiceToken` (Portal service secret, constant-time) → `req.service` ;
 6. `authenticatePublicKey` (public website API key) → `req.apiKeyContext` .
7. **Authorize (scope)** — `requireAuthz(Permission, getResource?)` → `can(user, action, resource)` → base-permission check + object-level policy + company boundary (ADMIN bypass).
8. **Validate** — `validateRequestBody(ZodSchema)` (strict shapes).
9. **Execute** — service → Prisma → MySQL.
10. **Respond** — JSON; errors mapped to `{ error, code }` (401/403/404/422/500).

5. Environments & Secrets

- `.env` is authoritative and **never committed**. `.env.example` exists.
- In production the server **fails fast at boot** if `JWT_ACCESS_SECRET` , `JWT_REFRESH_SECRET` , or `ENCRYPTION_KEY` are missing/too short.
- `ENCRYPTION_KEY` is required for AES-256-CBC encryption of customer KYC.
- AI configuration is disabled by default (`mock`).

6. Startup Seeding

On server start, it seeds the company, main branch, and a single technical **ADMIN** account (`RRH-ADMIN-001`) with a default password hash if employee count is zero. The Portal worker is launched but **disabled by default**. `prisma/seed.ts` seeds branches, company, and **all Roles/Permissions/RolePermissionsMatrix**.

CRM-IMPLEMENTATION-STATUS

Single authoritative implementation-status dashboard for RRH-CRMS.

1. Summary (top line)

CRM Module Implementation Status



Domain	Status	Notes
Authentication (JWT + refresh rotation)	●	<code>middleware/auth.ts</code> , <code>routes/auth.ts</code>
Authorization (RBAC + <code>can()</code> + data scope)	●	<code>authz/*</code> , <code>policies/*</code> , <code>middleware/authz.ts</code>
Company / tenant isolation	●	<code>dataScope.ts</code> + <code>can()</code> default; ADMIN bypass intended
Lead management	●	service + routes + assignment + activities
Opportunity pipeline	●	service + stage transition + pipeline metrics
Customer (+ KYC write)	●	service + routes; PAN/Aadhaar AES-256-CBC
Property + project + inventory	●	service + routes; publication + availability
Property matching engine	●	<code>matchingEngine.ts</code> weights exist; lead-scoped, no AI-search consumer & no public search endpoint
Site visits	●	book → verify → assign agent → complete
Bookings (+ property lock)	●	service + concurrency lock
Payments / installments	●	CRM record + verify; Portal sync callbacks
Collections	●	payments+installments implemented; no dedicated collections module

Domain	Status	Notes
Documents		upload/metadata/verify/archive/restore; KYC docs
KYC		encrypted PAN/Aadhaar; status; Portal submit callback
Tasks / SLA		task create/status + SLA derive + teams view
Escalation		task overdue auto-alerts; no general-purpose escalation engine
Notifications		in-app <code>Notification</code> + web-push subs; Portal notifications read-only
Complaints		Phase 14-1 lifecycle
Expense refunds		employee → accountant → MD → refunded
Attendance / QR		scan + status + proposals
Daily targets / daily reports		configure + submit + team view
Performance scoring		snapshots + score; leaderboard/team
Audit events		<code>AuditEvent</code> writes across key actions; admin read
Analytics / MD KPIs		<code>analytics.service.ts</code> + <code>/analytics/kpis</code> + <code>/md/executive-metrics</code>
Public property/project API		<code>routes/public.ts</code> (allowlist, API-key, brand-scoped)
Public lead capture		<code>POST /public/:brand/leads</code> (nullable creator)
Website search (public)		public list endpoints exist; no public structured-search endpoint
AI Search (Phase 17-A)		foundation only (mock provider, no HTTP route; tests only)
Customer Portal		integration groundwork (Phase 11) only; portal itself future
Channel Partner		OUT OF SCOPE (removed in Phase 10)

2. Cross-cutting status

Item	Status	Evidence
Model coverage in <code>prisma/schema.prisma</code>		30+ active models; only CP tables dropped

Item	Status	Evidence
Migrations present (MySQL)		20+ under <code>prisma/migrations/</code>
Test suite (Jest)		<code>tests/api/*.test.ts</code> incl. AI foundation tests
Playwright E2E		<code>playwright.config.ts</code> + 2 specs
TypeScript build/typecheck	(assumed healthy)	<code>run_tscheck.ps1</code> , <code>tsconfig.json</code>
Frontend (PWA) pages for all domains		<code>apps/web/src/components/**</code>
AI at runtime		no HTTP route, no provider config, disabled by default
<code>.env.example</code> completeness		only partial keys catalogued; <code>AI_*</code> absent

CRM-FEATURE-INVENTORY

Feature-by-feature matrix for RRH-CRMS. Columns: **Status**, **Backend**, **Frontend**, **Database**, **API**, **Tests**, **Notes**.

1. Core CRM

Feature	Status	Backend	Frontend	Database
Authentication		<code>routes/auth.ts</code> , <code>utils/jwt.ts</code>	<code>AuthContext.tsx</code>	<code>AuthSession</code>
Authorization / RBAC		<code>authz/*</code> , <code>policies/*</code>	role-gated UI	<code>Role</code> , <code>Perm</code>
Users / Employees		<code>routes/employees.ts</code>	PWA admin UI	<code>Employee</code>
Roles		seeded from shared	—	<code>Role</code> (11 ro

Feature	Status	Backend	Frontend	Database
Permissions		<code>packages/shared</code> (84)	—	<code>Permission</code>
Companies		<code>Company</code> model, seed	—	<code>Company</code>
Customers		<code>customer.service.ts</code>	PWA	<code>Customer</code>
Leads		<code>lead.service.ts</code>	PWA	<code>Lead</code> , <code>Lead</code>
Lead lifecycle		<code>lead.workflow.ts</code>	—	<code>Lead.status</code>
Lead assignment		<code>distributionService.ts</code>	—	<code>Lead.assigne</code>
Lead source / UTM		service	—	source, camp
Opportunities		<code>opportunity.service.ts</code> , <code>opportunity.workflow.ts</code>	PWA	<code>Opportunit</code>
Projects		<code>project.service.ts</code>	PWA	<code>Project</code>
Properties		<code>property.service.ts</code> , <code>property.workflow.ts</code>	PWA	<code>Property</code> , <code>PropertyPu</code>
Property inventory		service	—	<code>Property + B</code>
Site visits		<code>siteVisit.service.ts</code>	PWA	<code>SiteVisitB</code>
Bookings		<code>booking.service.ts</code>	PWA	<code>Booking</code>
Payments		<code>payment.service.ts</code>	PWA	<code>Payment</code>
Collections		payments + installments	PWA	<code>Installmen</code>
Documents		<code>document.service.ts</code>	PWA	<code>Document</code>
KYC		<code>kyc.service.ts</code>	—	<code>Customer KY</code>
Tasks		<code>routes/tasks.ts</code>	PWA	<code>Task</code>

Feature	Status	Backend	Frontend	Database
SLA		<code>task-sla.status.ts</code>	—	Task.target_c
Dashboards		analytics KPIs	<code>AnalyticsHub.tsx</code>	—
Search		<code>matchingEngine.ts</code>	—	—

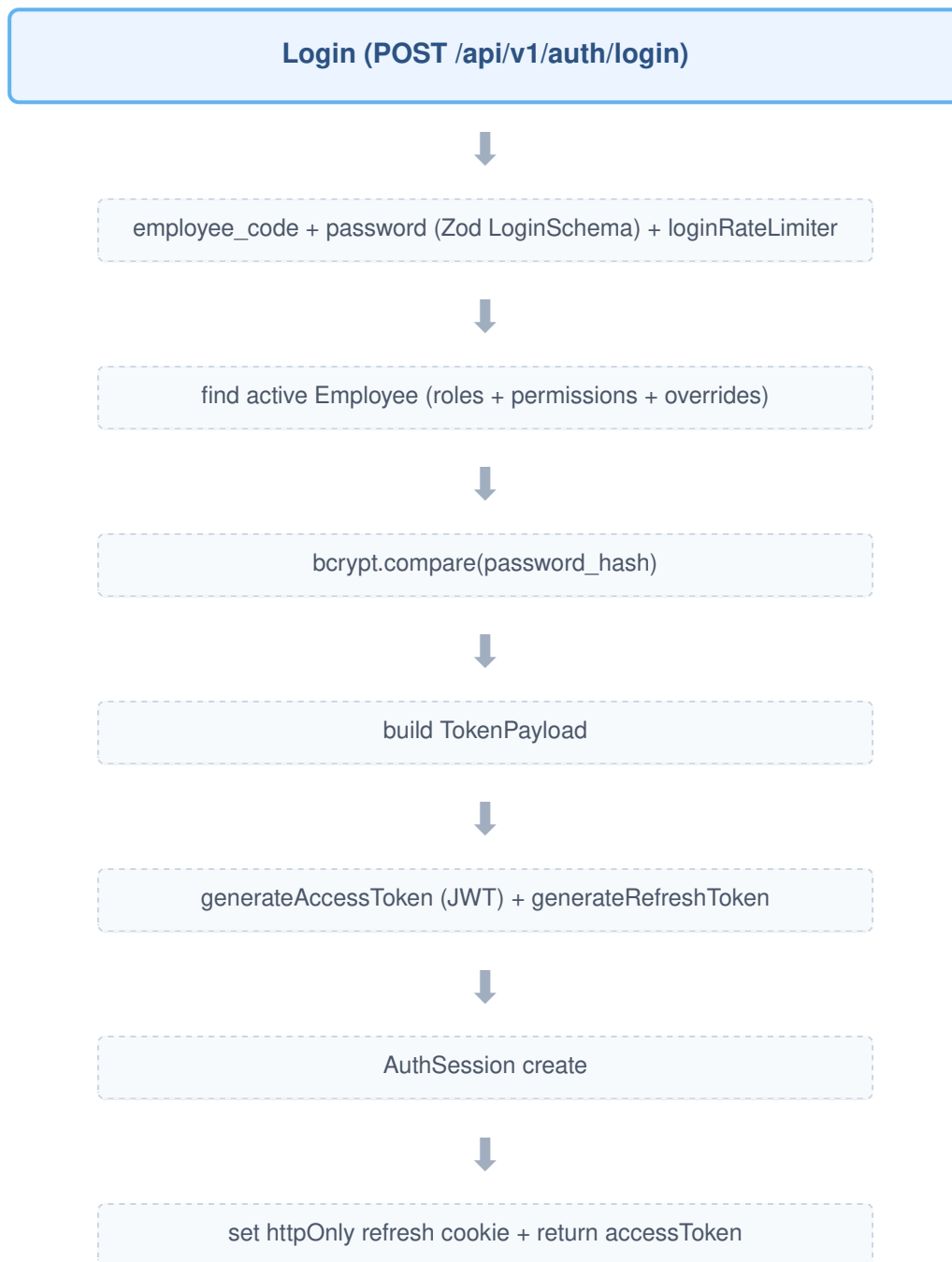
2. Status enums (verified)

Domain	Enum values (source: <code>prisma/schema.prisma</code>)
Lead.status	NEW, ASSIGNED, CONTACTED, QUALIFIED, SITE_VISIT_SCHEDULED, NEGOTIATION, WON, LOST, RECOVERED_TO_POOL
Opportunity.stage	PROSPECT_QUALIFIED, REQUIREMENT_CAPTURED, PROPERTY_SHORTLISTED, SITE_VISIT_PLANNED, SITE_VISIT_COMPLETED, PROPERTY_INTEREST_CONFIRMED, NEGOTIATION, BOOKING_INITIATED, BOOKED, DROPPED
Property.status	PENDING_VERIFICATION, PENDING_DM_POLISH, PENDING_MD_APPROVAL, LIVE, REJECTED, LOCKED, BOOKED, SOLD
Payment.status	PENDING, SUCCESS, FAILED, REFUNDED

CRM-WORKFLOWS

*Detailed, source-verified end-to-end workflows. Every flow lists **responsible role**, **API**, **database domain**, **authorization**, **status transitions**, and **business rules** as implemented.*

1. Authentication workflow



2. Lead workflow

Lead creation (POST /api/v1/leads OR POST /api/v1/public/:brand/leads)



validate LeadCreateSchema / PublicLeadCreateSchema



source (MANUAL_ENTRY | WEBSITE | ...) + UTM



status = NEW



assignment (auto performance-weighted / manual override / PM preference) → assigned_to_id



LeadActivity: LEAD_CREATED, ASSIGNED_TO_AGENT, ...



follow-up (Task creation) + SLA breach field (lead.sla_breach_at)

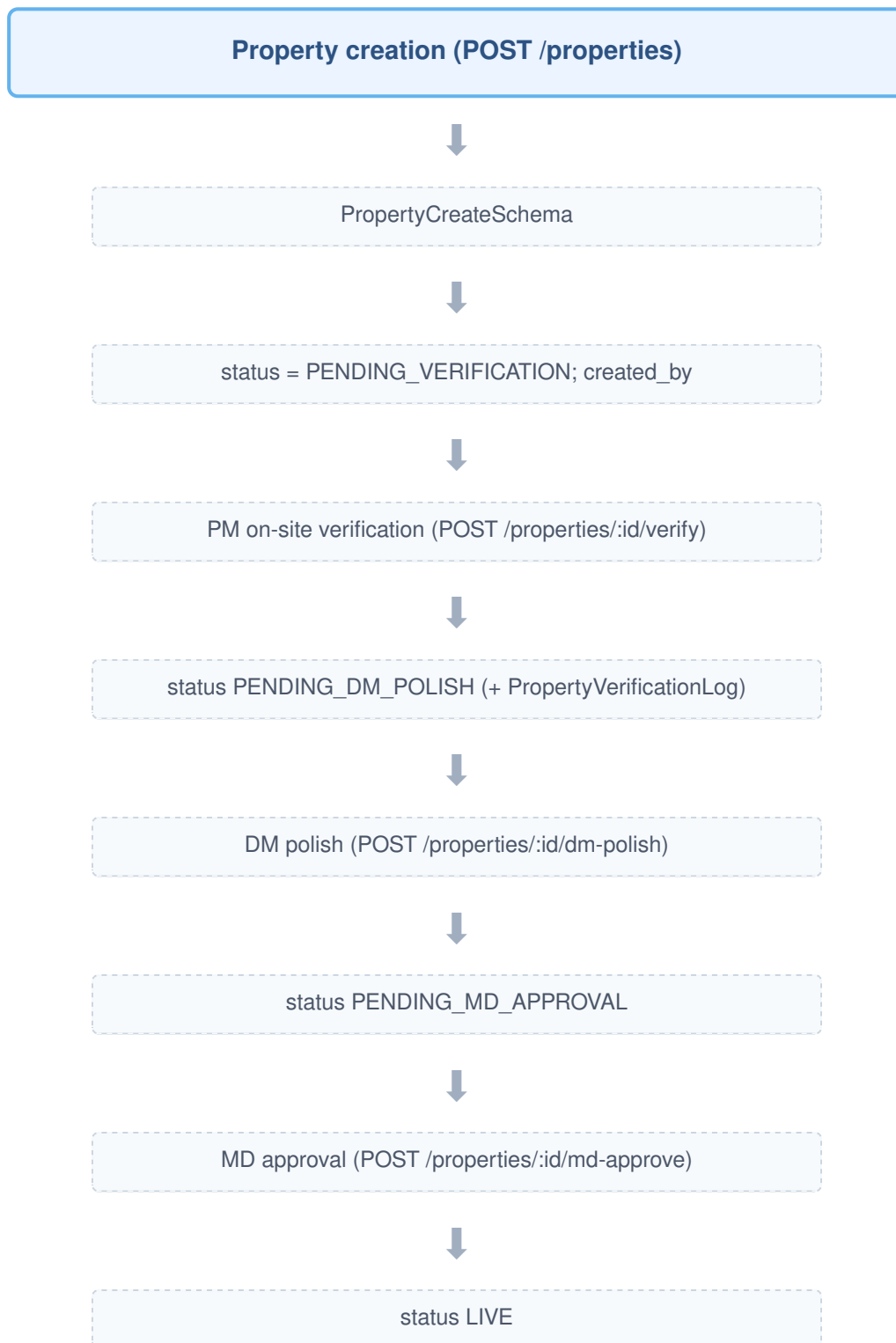


qualification (status QUALIFIED)



convert → Customer

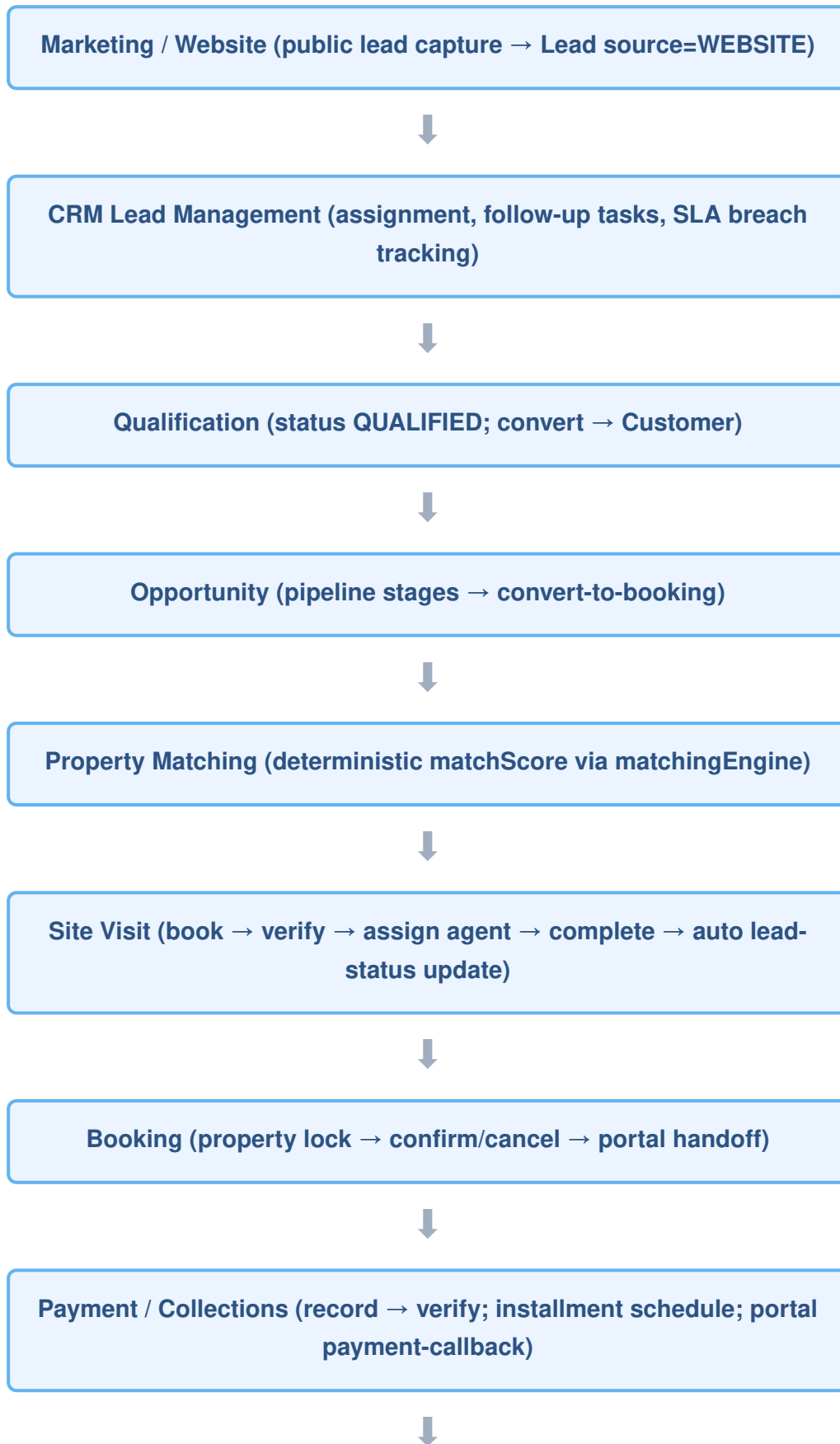
3. Property workflow



4. Site Visit workflow



5. Complete workflow map (master)



Documents / KYC (upload/verify; PAN/Aadhaar encrypted; portal kyc-callback)



Operations (tasks/SLA, expense refunds, complaints, attendance)



Reporting / Audit (daily reports, targets, performance, analytics KPIs, AuditEvent)

CRM-DATABASE-ARCHITECTURE

1. Active models (major) — purpose, relations, lifecycle, domain

Model	Purpose	Lifecycle	Domain
Company	Tenant (brand group RADHA_REAL_HOMES/ SONTHILLU)	active/deleted_at	Tenancy
Employee	Internal user	status ACTIVE/INACTIVE/SUSPENDED	Identity/HR
Lead	Sales enquiry	NEW...WON/LOST	Sales
Opportunity	Pipeline deal	stage enum (10)	Sales
Project	Development project	PLANNING/UNDER_CONSTRUCTION/ COMPLETED/CANCELLED	Inventory
Property	Inventory unit	PENDING_VERIFICATION...LIVE/ BOOKED/SOLD	Inventory
Booking	Sales booking	INITIATED/PENDING...COMPLETED/ CANCELLED	Commercial

Model	Purpose	Lifecycle	Domain
Payment	Payment record	PENDING/SUCCESS/FAILED/REFUNDED	Commercial
Customer	Customer master	ACTIVE/INACTIVE/BLACKLISTED	Customer/ KYC

CRM-API-CATALOG

1. Authentication APIs (/api/v1/auth)

Method	Endpoint	Auth	Purpose
POST	/auth/login	none	Login (rate-limited), issues access + refresh
GET	/auth/me		Current profile/permissions
POST	/auth/refresh	cookie	Rotate refresh token, return new access token

2. Properties (/api/v1/properties)

Method	Endpoint	Auth	Permission	Purpose
GET	/properties		PROPERTIES_READ	List (brand/status/project filters)
POST	/properties		PROPERTIES_CREATE	Create → PENDING_VERIFICATION
POST	/properties/:id/verify		PROPERTIES_VERIFY	PM on-site verification
POST	/properties/:id/md-approve		PROPERTIES_MD_APPROVE	MD approval → LIVE

3. Public website routes (/public)

Method	Endpoint	Auth	Purpose
GET	/public/:brand/properties	 key	Property listings (published, LIVE)
POST	/public/:brand/leads	 key	Create public lead

CRM-SECURITY-ARCHITECTURE

1. Authentication (authN)

Control	Status	Detail (source)
Password hashing		<code>bcryptjs</code> bcrypt (cost 12)
Access token (JWT)		<code>generateAccessToken</code> (<code>utils/jwt.ts</code>); short-lived Bearer
Refresh token		<code>generateRefreshToken</code> ; httpOnly cookie
Refresh rotation		every refresh mints a new token

2. Data-at-rest protections

Control	Status	Detail
KYC encryption		AES-256-CBC via <code>utils/crypto.ts</code> with <code>ENCRYPTION_KEY</code>
Passwords		bcrypt hash only, never stored plaintext

CRM-AI-ARCHITECTURE

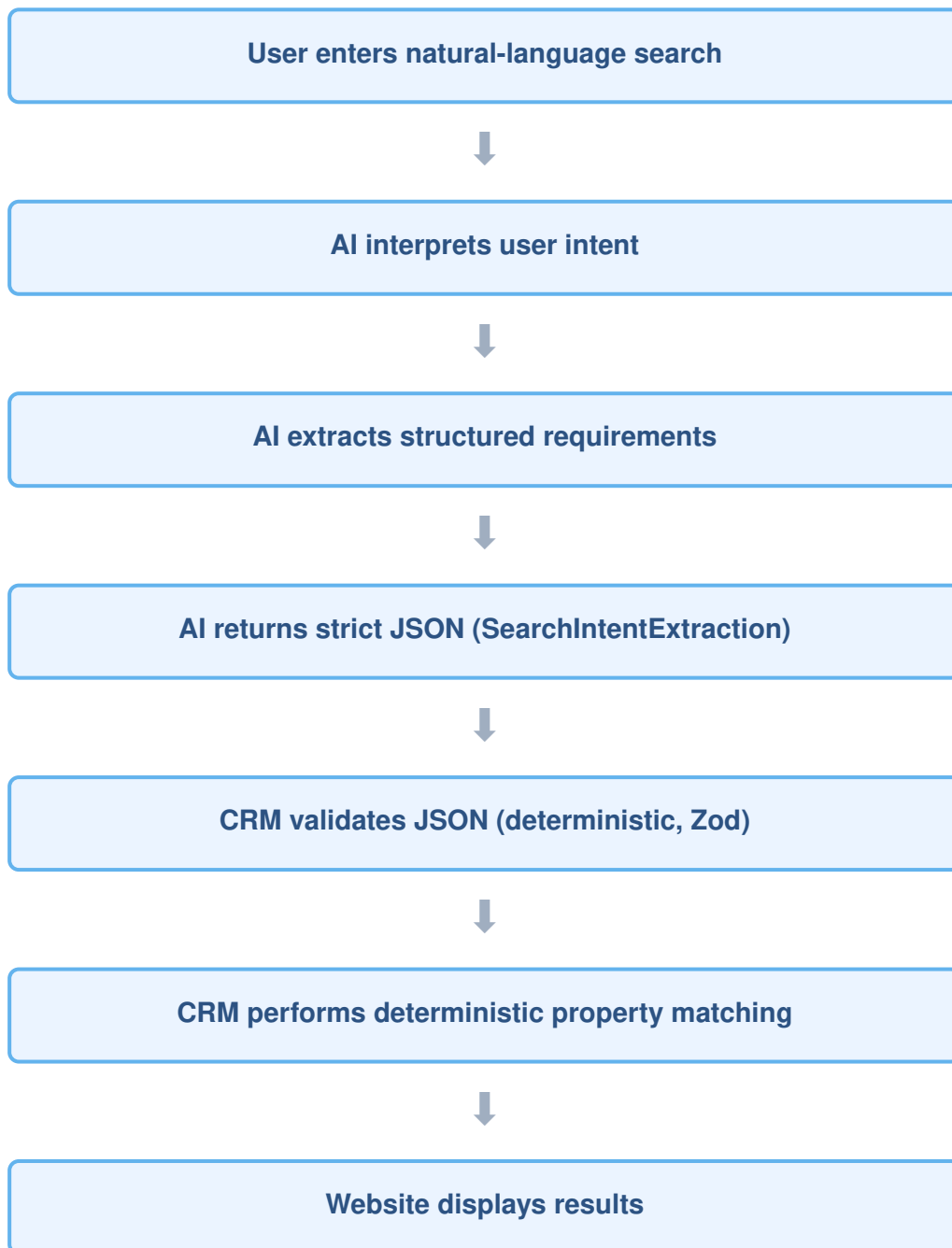
1. AI ROLE — interpretation layer, NOT a business-decision engine

The CRM AI is **not** a chatbot that recommends properties. Its sole role is:

Natural-language requirement extraction and structured search-query generation.

The AI **must NOT**: - recommend properties / tell users what to buy; - rank properties or calculate match percentage; - modify property data or any CRM data; - execute business workflows.

2. Intended AI Search workflow



CRM-WEBSITE-INTEGRATION

1. Responsibility boundary

The **website** is the **user-facing search & presentation interface**; the **CRM** is the **business and data engine**.

Responsibility	Owner
Public presentation & SEO	Website
Natural-language → SearchIntent extraction	CRM AI
SearchIntent validation	CRM
Property matching & ordering	CRM
Booking / payment / collection decisions	CRM

2. Public APIs actually available (verified)

All under `/api/v1/public/:brand/...` authenticated by the **public API key**. Rate-limited. Public response allowlists (`PUBLIC_PROPERTY_SELECT`) exclude internal fields: **status**, **GPS coordinates**, **company_id**, **branch_id**, **seller info**.

CRM-V2-ROADMAP

1. EXPLICITLY OUT OF SCOPE

Area	Status	Evidence / notes
Channel Partner (CP)	☐ OUT OF SCOPE	Phase 10 Packet 1 excision; Phase 13 PERMANENTLY EXCISED.
Autonomous AI agents	☐ OUT OF SCOPE	AI is an interpretation layer, not an agent framework.

Area	Status	Evidence / notes
Arbitrary SQL through AI	☐ OUT OF SCOPE	AI emits structured <code>SearchIntent</code> only.

2. FUTURE / V2 — Classification

Status	Meaning
☒ Planned	named in an authoritative roadmap as a future phase
☒ Proposed	reasonable extension, but not yet named/approved in roadmap

AI — Future (V2) items

Item	Status	Notes
Real provider adapters (OpenAI/OpenRouter/local)	☒ Proposed	SDKs declared but unused
HTTP AI-search endpoint	☒ Proposed	foundation contract exists; no route